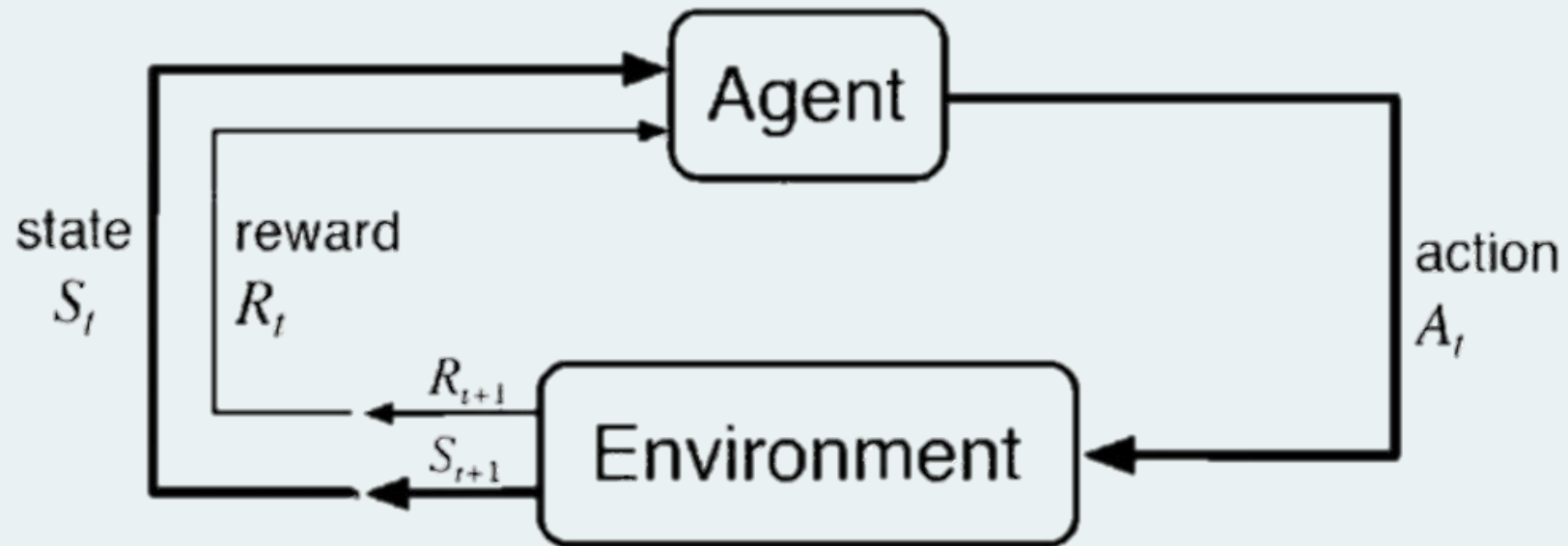


Function Approximation in Reinforcement Learning

Presented by: Reem Khattar

Reinforcement Learning



Agenda

1. The Curse of Dimensionality
2. What is Function Approximation?
3. Types of Function Approximators
4. Training Function Approximators
5. Key Algorithms
6. Challenges and considerations
7. Addressing the Challenges
8. Applications of Function Approximation

The Curse of Dimensionality



The Curse of Dimensionality

- Memory explosion
- Computational bottleneck
- Data scarcity
- Inability to handle continuous state spaces
- Generalization limitations



Overcoming the Curse: Function Approximation

Function Approximation

Representing value functions or policies:

~~tables~~ → parametrized functions

$$V(s), Q(s, a) \rightarrow V_{\theta}(s), Q_{\theta}(s, a)$$

- Represent large or continuous state spaces
- Generalize to unseen states
- Reduce memory and computational demands

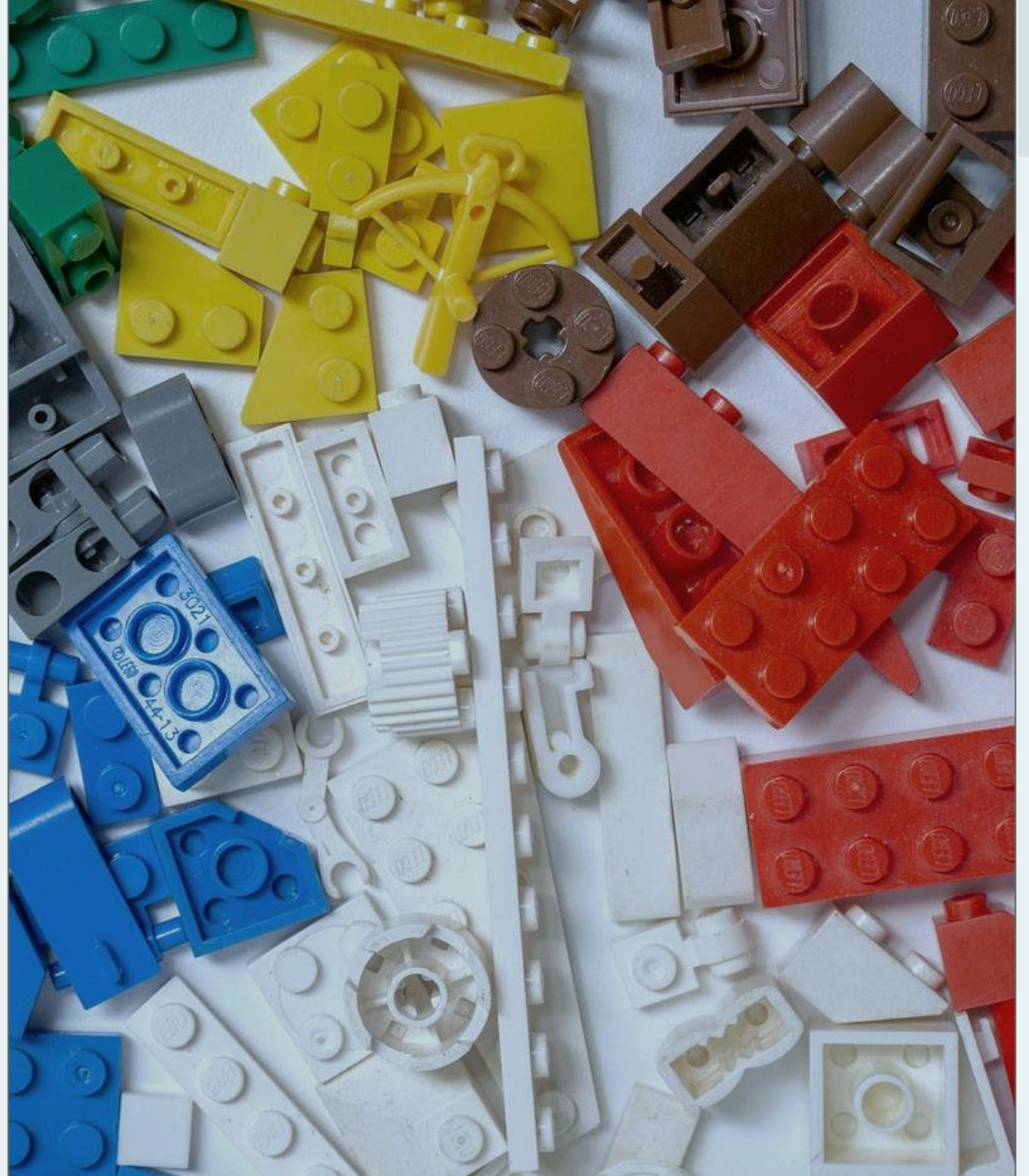
Function Approximation

Key ideas:

- Extracting features
- Choosing function approximator (linear functions, neural networks, ...)
- Learning

We go from **storing explicit values** for every state to **learning a compact representation** that captures the underlying structure of the problem.

Feature Construction: The Art of Representation



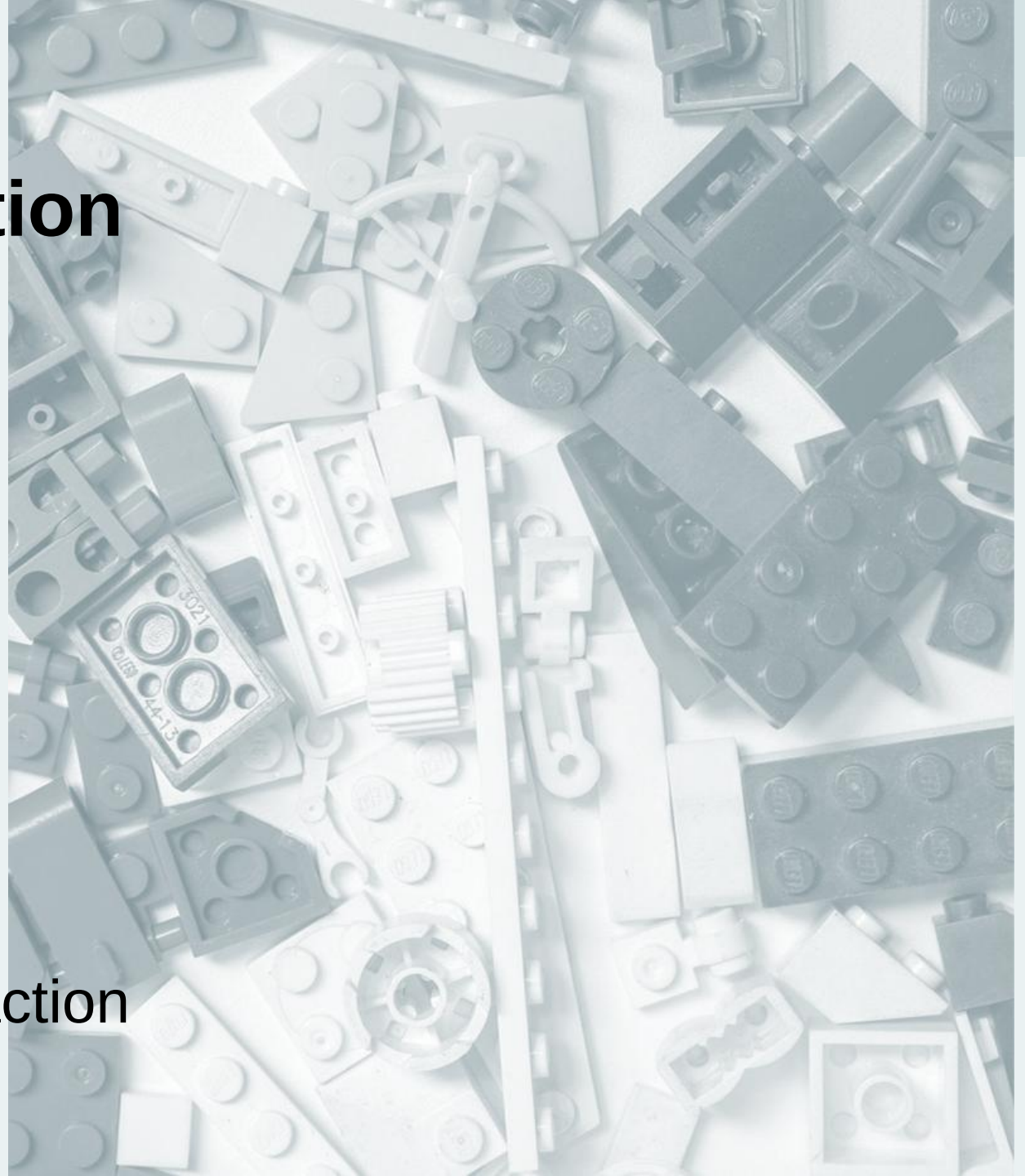
Feature Construction

Key Considerations:

- Relevance
- Discriminative power
- Compactness

Common Techniques:

- Hand-crafted features
- Automatic feature extraction



Examples of Feature Construction

Robot navigation:

- Distances to obstacles
- Robot velocity
- Goal direction

Game playing:

- Piece positions
- Game score
- Board configuration

...



Types of Function Approximators

1. Linear Function Approximation

Represents the value function as a linear combination of features, each feature weighted by a coefficient.

$$Value = w_1f_1 + w_2f_2 + \cdots + w_nf_n$$

Strengths:

- Simple
- Computationally efficient
- Convergent under certain conditions

Weaknesses:

- Can only model linear relationships between features; may not be expressive enough for complex problems

2. Polynomial Function Approximation

Expansion of linear function approximation, incorporates higher-order terms.

$$Value = w_{11}f_1 + w_{21}f_2 + \cdots + w_{12}f_1^2 + w_{22}f_2^2 + \cdots$$

Strengths:

- Can model some non-linear relationships

Weaknesses:

- Prone to overfitting
- Sensitive to the curse of dimensionality

3. Neural networks

Powerful function approximators capable of learning highly non-linear relationships. Effective for high-dimensional state spaces.

Strengths:

- Highly expressive
- Can learn complex patterns
- Effective in high dimensional spaces

Weaknesses:

- Can be difficult to train
- Prone to overfitting
- May require substantial data and computational resources

4. Decision Trees and Ensemble Methods:

Decision trees partition the state space into regions and assign a value or action to each region.

Ensemble methods combine multiple decision trees for improved performance.

Strengths:

- Can handle non-linearity
- Relatively interpretable

Weaknesses:

- Prone to overfitting
- May not generalize well in continuous state spaces

5. Radial Basis Function (RBF) Networks:

They use radial basis functions as activation functions. These functions have centers in the state space, allowing the network to learn localized patterns in the value function.

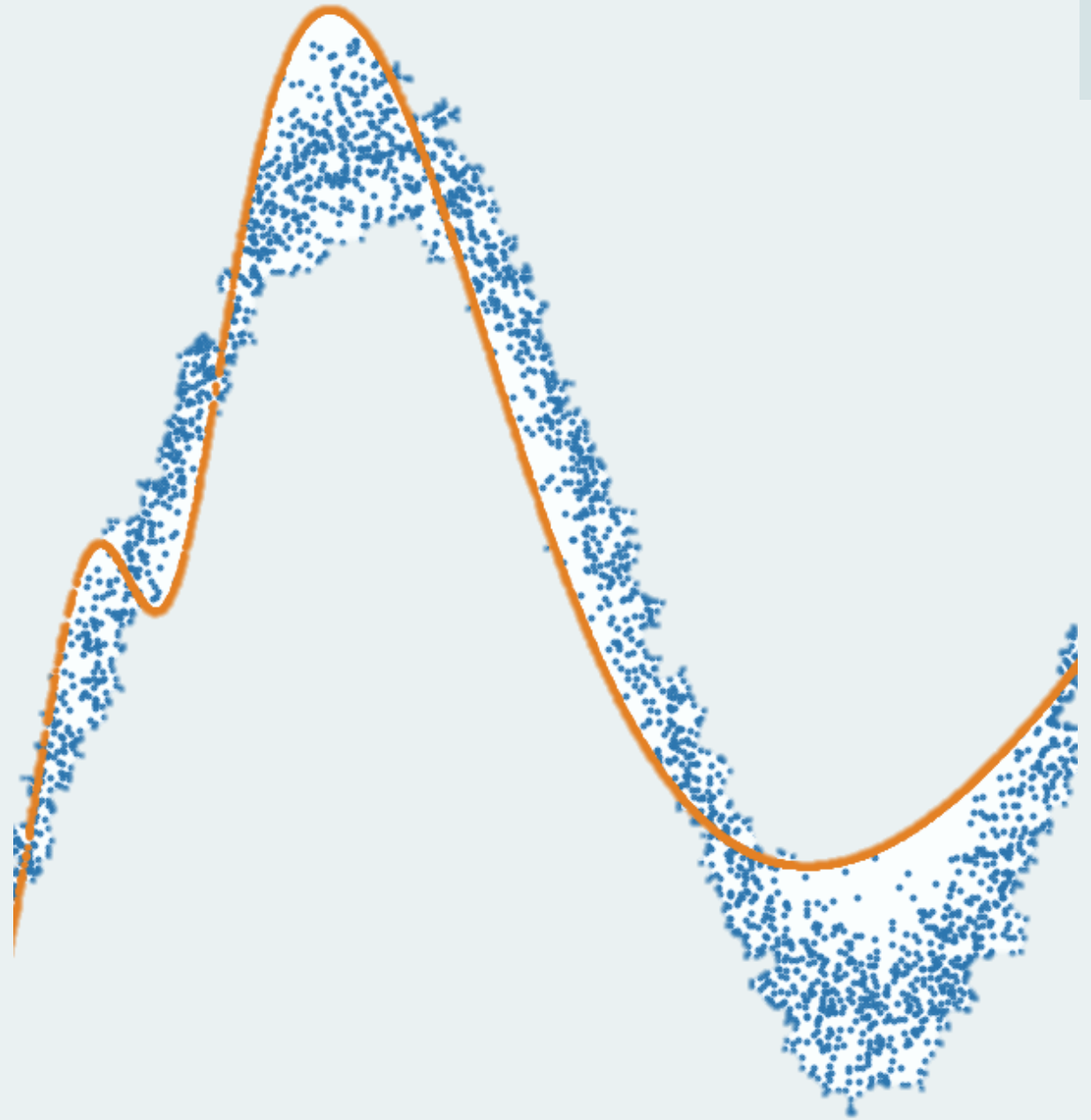
Strengths:

- Effective in continuous state spaces
- Can handle non-linearity

Weaknesses:

- The choice of centers and widths of the RBFs can impact the performance

Training Function Approximators



Training Function Approximators

1. Choose a function approximator
2. Choose a loss function
3. Choose a target value
4. Update the parameters of the function approximator

Training Function Approximators

- Input: state or state-action pair
- Output: estimated value
- Training data: experiences collected by the agent as it interacts with the environment
 - States visited
 - Actions taken
 - Rewards received
 - Next states

Training Function Approximators

Objective: minimize the difference between the value estimated by the approximator and the target value

Common Loss Function: MSE

Gradient descent: adjust the parameters of the function approximator in a way that minimizes the loss function.

Training Function Approximators

Calculating Target Values:

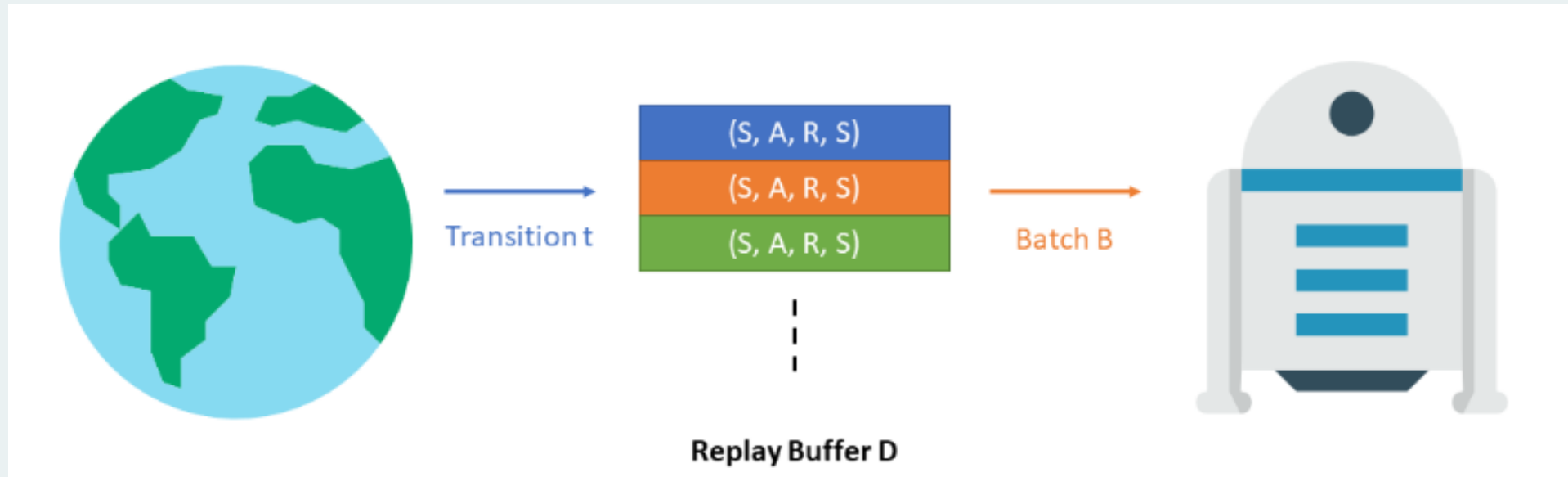
- Monte Carlo methods:
 - Target: true return from a complete episode
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$
- Temporal Difference methods:
 - Target: combines immediate reward and estimated value of the next state
$$G_t = R_{t+1} + \gamma V(S_{t+1})$$
- n-step methods: a combination of Monte Carlo and TD methods.

Challenges and Considerations

Challenges and Considerations

- Stability and convergence not guaranteed
- The Deadly Triad
 - Function approximation
 - Off-policy learning
 - Bootstrapping
- Overfitting

Addressing the Challenges: Experience Replay



Experience Replay

- Store past experiences (state, action, reward, next state) in a buffer
- Randomly sample experiences from the buffer for training

Benefits:

- Helps break correlation and make data more i.i.d
- Reduces Variance
- Improves sample efficiency (reuses past experiences)
- Facilitates off-policy learning

Prioritized Experience Replay

- Variation of standard experience replay
- Experiences are weighted based on their importance

Benefit:

- Further improved efficiency

Applications of function approximation

Applications of function approximation

- Robotics control
 - Neural Networks function approximators can learn complex mappings from these high-dimensional, continuous states to appropriate motor commands
- Game playing
 - Function approximation, particularly using deep neural networks helped develop agents that can master these games
- Finance
 - Function approximators can learn to adapt trading strategies based on historical market data and current market conditions, for better investment decisions

Thank you